12) a) Implement echo client server using TCP/UDP Sockets.

Server Program:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main()
{
    int client;
    char buffer[1024] = "Hello Server";
    struct sockaddr_in addr;

    client = socket(AF_INET, SOCK_STREAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(8080);
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(client, (struct sockaddr*)&addr, sizeof(addr));

    send(client, buffer, strlen(buffer) + 1, 0);

    recv(client, buffer, sizeof(buffer), 0);

    printf("Echo from Server: %s\n", buffer);

    close(client);

    return 0;
}
```

Client Program:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main()
{
    int server, client;
    char buffer[1024];
    struct sockaddr_in addr;
    socklen_t len = sizeof(addr);

    server = socket(AF_INET, SOCK_STREAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(8080);
    addr.sin_addr.s_addr = INADDR_ANY;

    bind(server, (struct sockaddr*)&addr, sizeof(addr));
    listen(server, 1);

    printf("Server waiting...\n");

    client = accept(server, (struct sockaddr*)&addr, &len);

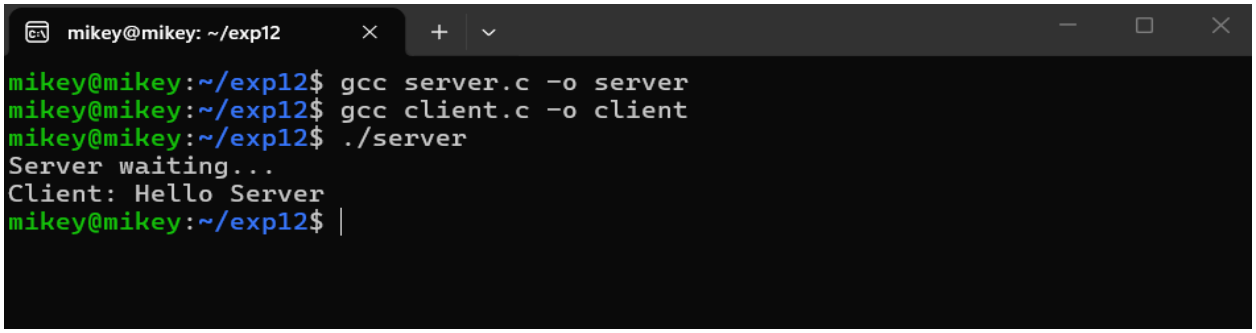
    recv(client, buffer, sizeof(buffer), 0);
    printf("Client: %s\n", buffer);

    send(client, buffer, strlen(buffer) + 1, 0);

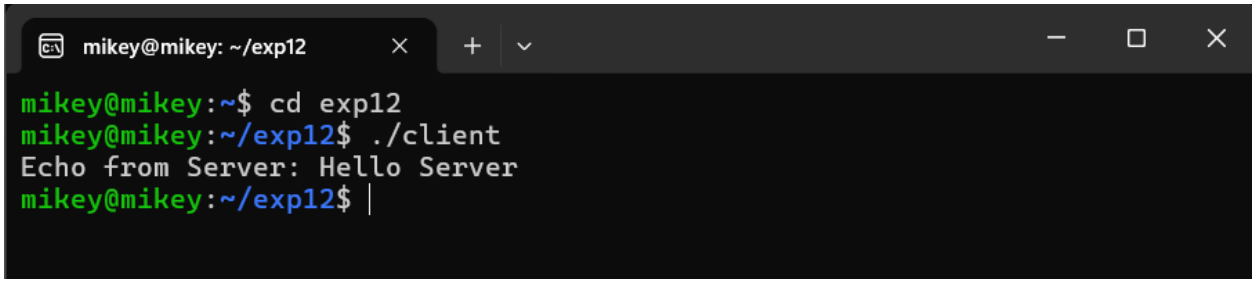
    close(client);
    close(server);

    return 0;
}
```

Server Terminal:



Client Terminal:



12) b)Implement chat client server using TCP/UDP sockets.

Server Program:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main()
{
    int client;
    char buffer[1024];
    struct sockaddr_in addr;

    client = socket(AF_INET, SOCK_STREAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(8080);
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(client, (struct sockaddr*)&addr, sizeof(addr));

    while (1)
    {
        printf("Client: ");
        fgets(buffer, sizeof(buffer), stdin);
        buffer[strcspn(buffer, "\n")] = '\0';

        send(client, buffer, strlen(buffer) + 1, 0);

        if (strcmp(buffer, "exit") == 0)
            break;

        recv(client, buffer, sizeof(buffer), 0);
        printf("Server: %s\n", buffer);

        if (strcmp(buffer, "exit") == 0)
            break;
    }

    close(client);

    return 0;
}
```

Client Program:

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main()
{
    int server, client;
    char buffer[1024];
    struct sockaddr_in addr;
    socklen_t len = sizeof(addr);

    server = socket(AF_INET, SOCK_STREAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(8080);
    addr.sin_addr.s_addr = INADDR_ANY;

    bind(server, (struct sockaddr*)&addr, sizeof(addr));
    listen(server, 1);

    printf("Waiting for client...\n");

    client = accept(server, (struct sockaddr*)&addr, &len);

    while (1)
    {
        recv(client, buffer, sizeof(buffer), 0);
        printf("Client: %s\n", buffer);

        if (strcmp(buffer, "exit") == 0)
            break;

        printf("Server: ");
        fgets(buffer, sizeof(buffer), stdin);
        buffer[strcspn(buffer, "\n")] = '\0';

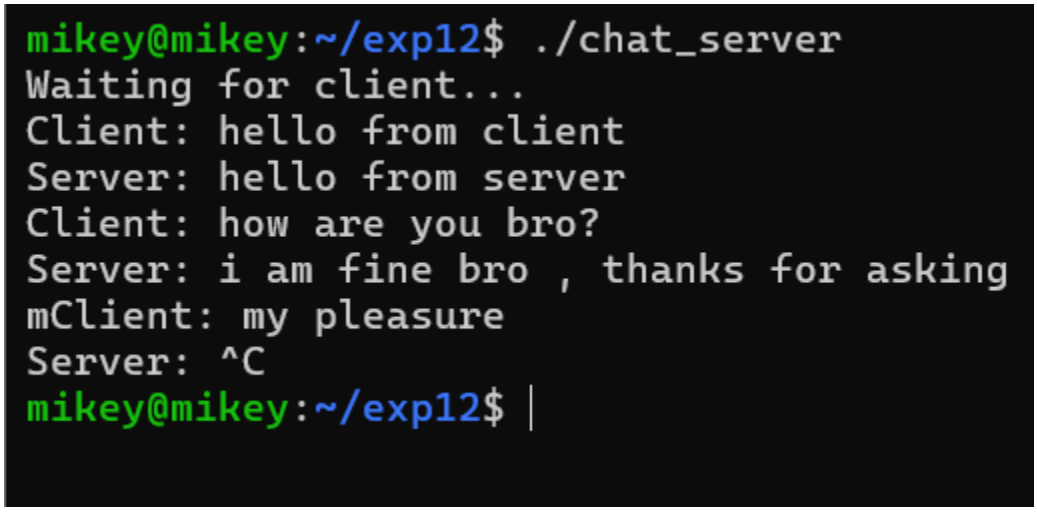
        send(client, buffer, strlen(buffer) + 1, 0);

        if (strcmp(buffer, "exit") == 0)
            break;
    }

    close(client);
    close(server);

    return 0;
}
```

Server Terminal:



Client Terminal:

